Doing It Again: The `while` Loop

Class 8 — In-Class Work-Along Sheet (Part 1 — the loop, discovered)

A `while` loop runs a block of code **over and over, as long as a condition is True**.

You write the action **once**; the loop performs it as many times as the condition allows.

How to use this sheet: for every "predict" box, write your answer **before** you run the code. The point is to catch the gap between what you *think* happens and what *actually* happens — that gap is where the learning is. The ⟳ symbol marks a question that reaches back to an earlier class.

Part 1 — Meeting the Loop

Q1 — Write it the painful way first

(a) Using only `print()` statements (no loop yet), write code that prints the numbers 1 to 5, one per line:

```
_____________________________________________________________
_____________________________________________________________
```

(b) Now imagine the task was **print 1 to 1000**. In one sentence, why is the approach in (a) a bad idea?

```
_____________________________________________________________
```

(c) Look at your five lines from (a). What is **identical** in every line, and what **changes** from one line to the next?

```
Identical: _____________________     Changes: _____________________
```

Q2 — The loop that replaces all of it

Predict first. Write down every line you think this prints:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
Predicted output:                    Actual output (after running):
_____________                        _____________
```

(a) The body has a single `print(i)` — just one line. How many lines of output did it produce? _____

(b) If you changed `i <= 5` to `i <= 1000`, how many lines would print? _____ How many lines of code did you have to change to do that? _____

(c) In your own words, what does the word `while` mean here?

```
_____________________________________________________________
```

Q3 — When is the condition checked?

Predict first, then run. What does this print?

```
n = 0
while n > 0:
    print("body runs")
    n -= 1
print("done")
```

```
Predicted: _____________        Actual: _____________
```

(a) How many times did the body (`print("body runs")`) execute? _____

(b) Explain why, using the word **"before"**:

```
_____________________________________________________________
```

(c) Fill in the rule: A `while` loop checks its condition **at the (top / bottom)** of the loop, **(before / after)** each pass — including the **(first / last)** one. *(circle one in each)*

Q4 — Trace by hand (no running!)

Step through this loop **on paper**. Fill one row per pass. In each row write the value of `i` as it stands when the condition is checked (before that pass's body runs), whether the condition is True or False, and what (if anything) prints.

```
i = 2
while i <= 8:
    print(i)
    i += 2
print("i at the end:", i)
```

```
 Pass | i at check | i <= 8 ?  | prints?
 -----+------------+-----------+---------
  1   |     2      |   True    |   2
  2   |  _______   |  _______  |  _______
  3   |  _______   |  _______  |  _______
  4   |  _______   |  _______  |  _______
  5   |  _______   |  _______  |  _______
```

(a) What is the value of `i` printed by the final line? _____

(b) It is **not** 8. Explain in one sentence why the loop variable ends up *past* the boundary:

```
_____________________________________________________________
```

Q5 — Name the four parts

Every well-formed `while` loop has four jobs. Label each line below with its job — choose from: **INITIALISE · CONDITION · BODY · UPDATE**.

```
count = 10                  # _____________
while count > 0:            # _____________
    print(count)            # _____________
    count -= 1              # _____________
```

(a) Which one of the four, if you delete it, would make this loop **never stop**? _____

(b) Which one decides whether the loop runs **zero** times? _____

Q6 — Generate before you're told

You have **not** been shown a countdown loop. Build one anyway, using the four parts as your guide.

Write a `while` loop that prints `5 4 3 2 1` (each on its own line) and then prints `liftoff`:

```
_____________________________________
_____________________________________
_____________________________________
_____________________________________
_____________________________________
```

(a) What did you initialise your variable to? _____ **(b)** What is your condition? _____ **(c)** What is your update? *(hint: this loop counts DOWN)* _____ **(d)** Predict the value of your variable *after* the loop ends, then check: _____

Spaced recall — reach back ⟳

These don't use loops. They keep earlier ideas warm, and you'll need them next part.

Q7 — ⟳ from Class 3 (modulo & floor division)

Without running, predict:

(a) `17 % 5` = _____ **(b)** `17 // 5` = _____

(c) In one line, what does `n % 10` give you about the number `n`? And what does `n // 10` do to it?

```
n % 10  → _____________________     n // 10 → _____________________
```

(You'll use exactly this pair next part to pull a number apart digit by digit inside a loop.)

Q8 — ⟳ from Class 6 / 7 (conditions & the boolean a loop depends on)

A `while` loop keeps going **as long as its condition is True**. So you must be able to read a condition cold.

For `x = 7`, predict True/False for each (watch precedence — ⟳ Class 7):

(a) `x > 0` → _____ **(b)** `x % 2 == 0` → _____ **(c)** `x > 0 and x < 10` → _____ **(d)** `not x > 10` → _____

(e) A loop header reads `while x % 2 == 0:`. For `x = 7`, does the body run even once? _____ Why?

```
_____________________________________________________________
```

Q9 — Books closed — free recall

Close this sheet. From memory, write:

(a) The one job a `while` loop does that an `if` cannot:

```
_____________________________________________________________
```

(b) The four parts of a `while` loop, in order:

```
1. _____________   2. _____________   3. _____________   4. _____________
```

(c) A `while` loop's condition is False the *first* time it is checked. How many times does the body run, and why?

```
_____________________________________________________________
```

Part 2 — When Loops Go Wrong

Two classic bugs, and both come straight from Part 1's rule (*condition checked at the top, every pass*):

a loop that **never stops** (forgotten update), and a loop that stops **one step wrong** (off-by-one).

Q10 — The loop that won't quit

Predict first. What does this print — and for how long?

```
i = 1
while i <= 3:
    print(i)
```

```
Predicted: _________________     Actual: _________________
```

(a) Which of the four parts (initialise / condition / body / update) is **missing**? _____

(b) Because that part is missing, the value of `i` stays at _____ forever, so the condition `i <= 3` is always _____.

(c) How do you stop this in the terminal? _____ What verdict would an online judge (e.g. Codeforces) give instead? _____

Q11 — `while True` + `break`: escaping on purpose

Trace this. You type, in order: `hi`, then `go`, then `quit`.

```
while True:
    cmd = input("cmd> ")
    if cmd == "quit":
        print("bye")
        break
    print("you said:", cmd)
```

Write the full output (include the `cmd>` prompts):

```
_____________________________________________________________
_____________________________________________________________
_____________________________________________________________
```

(a) The condition is literally `True`. So what is the *only* way this loop ever ends? _____

(b) The special value that signals "stop" (here, `quit`) has a name — the s_____ value.

Q12 — Off-by-one: the silent bug

You want to print 1 to 10. **Predict each** before running.

Version A

```
i = 1
while i < 10:
    print(i, end=" ")
    i += 1
```

Predicted: _____ Actual: _____

Version B

```
i = 1
while i <= 10:
    print(i, end=" ")
    i += 1
```

Predicted: `_____` Actual: `_____`

(a) One of them is missing a number. Which version, and which number? _____

(b) Complete the rule: to **include** the endpoint `n`, use `while i __ n`; to **exclude** it, use `while i __ n`.

(c) This bug does *not* crash. So what is the only way you'll catch it? _____

Q13 — Diagnose & fix (which bug is it?)

For **each** loop: first decide *which kind* of bug it has — **infinite loop** or **off-by-one** — then write the one-line fix. (Deciding the kind first is the whole skill.)

Loop A — meant to count down from 5 to 1:

```
n = 5
while n > 0:
    print(n)
```

Bug kind: _____ Fix: _____

Loop B — meant to print the squares 1^2, 2^2, 3^2, 4^2, 5^2:

```
i = 1
while i < 5:
    print(i * i)
    i += 1
```

Bug kind: _____ Fix: _____

Q14 — Generate before you're told

Write a `while True` loop that keeps asking for a password. While the input is wrong it prints `wrong, try again` and asks again. When the user types `open sesame`, it prints `unlocked` and stops.

```
_____
_____
_____
_____
_____
```

(a) What is your sentinel (the value that ends the loop)? _____ **(b)** Where did you put the `break` — and why must it be inside an `if`? _____

Spaced recall — reach back ⟳

Q15 — ⟳ Class 7 (precedence) + ⟳ Part 1 (end-state)

(a) A loop guard reads `while not count == 0:`. Using Class 7 precedence — does `not` or `==` bind **tighter**? _____ Add parentheses to show how Python actually groups it:

```
while not count == 0:   →   while _____________________ :
```

(b) ⟳ From Part 1 — this countdown runs cleanly to the end. What is `n` *after* the loop finishes?

```
n = 5
while n >= 1:
    print(n)
    n -= 1
```

`n` at the end = _____

Q16 — Books closed — free recall (Part 2)

Close the sheet. From memory:

(a) The two classic loop bugs, and the one-word symptom of each:

```
1. ____________________  → symptom: ____________________
2. ____________________  → symptom: ____________________
```

(b) `while True` never stops by itself. What keyword escapes it, and where do you usually place it?

```
_____________________________________________________________________
```

Coding Problems — On the LMS

Submit these on the **LMS code judge** — not on this sheet. Read input from stdin, print to stdout.

Everything you need you already have: a `while` loop, `if` / `else`, `%` and `//` (Class 3), and now `break`.

Solve them **in order** — each adds one new idea to the last. Don't peek at a solution; the struggle is the point.

P1 — Multiplication Table `[LMS: loops-mult-table]`

Given an integer `n`, print its multiplication table from 1 to 10. Each line reads `n x i = product`.

```
Input:   19
Output:  19 x 1 = 19
         19 x 2 = 38
         19 x 3 = 57
            ... (one line per i) ...
         19 x 10 = 190
```

Tools you have: a counter loop running from 1 to 10.

P2 — All Factors of a Number `[LMS: loops-factors]`

Given an integer `n`, print every positive integer that divides `n` exactly (its factors), in increasing order, one per line.

```
Input:   12              Input:   19
Output:  1               Output:  1
         2                        19
         3
         4
         6
         12
```

Tools you have: loop `i` from 1 to `n`; the test "does `i` divide `n`?" is one you met in Class 3.

P3 — Factorial `[LMS: loops-factorial]`

Given an integer `n` (with `n >= 0`), print `n!` — that is `1 x 2 x 3 x ... x n`. By definition `0! = 1`.

```
Input:   5         Input:   0         Input:   10
Output:  120       Output:  1         Output:  3628800
```

Tools you have: a loop, and a variable that builds up the answer across passes. Think hard about one thing — what should that variable start at, and why is starting it at 0 a disaster here?

P4 — Prime or Not `[LMS: loops-prime]`

Given an integer `n` (with `n >= 1`), print `prime` if `n` is a prime number, otherwise print `not prime`. (A prime has no divisors other than 1 and itself; 1 is *not* prime.)

```
Input:   7         Input:   20            Input:   2         Input:   1
Output:  prime     Output:  not prime     Output:  prime     Output:  not prime
```

This one is the climb. Two sub-problems you must crack yourself:

1. As soon as you find a divisor, you know the verdict — there's no point checking further. What Part 2 tool stops a loop early?

2. After the loop ends, your program has to remember whether it ever found a divisor, so it can print the right answer. There is no Python feature that does this for you — you'll have to invent it.

After P4 — what did you just invent?

(a) In your P4 solution you needed a variable to remember "have I found a divisor yet?". What did you name it? _____ Is that name a Python keyword (like `while` or `break`), or an ordinary variable *you* chose? _____

(b) `while`, `if`, and `break` are part of the Python *language* — its designers built them in and you looked them up. Did you look up "remember a yes/no result across a loop" anywhere in Python? _____ Could you use the very same idea in C++, Java, or any other language? _____

(c) The thing you invented in P4 has a name — a **flag**. But Python has never heard of it: there is no `flag` keyword and no "flag feature." It is a **technique** — a problem-solving idea that lives in *your* head and works in *every* language. In one sentence of your own, what is the difference between a **language construct** (like `break`) and a **technique** (like the flag)?

_____
_____

The distinction that makes you a programmer: a construct is **given** to you by a language — you learn it once, per language. A technique you **invent** as a problem solver, and carry into every language you ever touch. Knowing `while` makes you a Python user; inventing the flag makes you a problem solver. Python knew nothing about how to test for a prime — you did.

Beyond the Judge — Three Harder Skills

On a coding portal the problem is always complete and the code always visible. Real work isn't like that. These three problems train the skills the judge can't: spotting a gap in a spec, debugging code you can't fully see, and using a tool nobody taught you.

S1 — Is the question even complete?

An interviewer often states a problem *vaguely on purpose*, to see whether you notice what's missing before you start typing. Here is one such statement:

> **`BIGGEST`** — *"Read some numbers and print the biggest one."*

This statement is **not complete**. Before writing a single line of code, write **three questions** you would ask to remove the ambiguity:

```
Q(i)  : _____________________________________________________
Q(ii) : _____________________________________________________
Q(iii): _____________________________________________________
```

Now — and only now — read the clarified spec:

```
Clarified spec
--------------
  - First line is a count K        (K >= 1)
  - The next K lines: one integer each
  - The integers MAY be negative
  - Print the maximum value
```

(a) Did any of your three questions catch that the numbers can be **negative**? `_____`

(b) Why does "numbers may be negative" change how you set up your `max` variable — i.e. why is starting it at 0 a bug here?

```
_____________________________________________________________
```

S2 — The hidden line (debugging from a trace)

Reading an error is a skill, and being unable to see all the code is normal — that's most bug reports. Below, a program reads a number each pass and prints a result. **One line is hidden from you.** Here is the actual run, and the error it ended with:

```
iteration = 1
while iteration <= 5:
    x = int(input())
    ____________________________________   # <-- THIS LINE IS HIDDEN
    iteration = iteration + 1
```

```
Console session
---------------
6
Iteration 1 -> 20.0
3
Iteration 2 -> 40.0
10
Iteration 3 -> 12.0
?                  (a 4th number was entered, but it is hidden from you)
Traceback (most recent call last):
  File "main.py", line 4, in <module>
    < hidden line >
ZeroDivisionError: division by zero
```

(a) Look at the pairs *(input, result)*: `(6, 20.0)`, `(3, 40.0)`, `(10, 12.0)`. What single fixed number is being divided by `x` on every pass? `_____`

(b) Using that, write what you think the hidden line (line 4) was:

```
_____________________________________________________________
```

(c) In your own words, what does `ZeroDivisionError` actually mean here?

```
_____________________________________________________________
```

(d) What must the **4th number** entered have been, and how do you know?

```
_____________________________________________________________
```

S3 — Use a tool you were never taught

A warning: this one is hard, and it uses things you have **not** been taught yet — classes and methods. That is the whole point. You do not need to understand how the tool works *inside*; you only need to trust its **contract** — what it promises to do. Reading an unfamiliar tool and using it anyway is the real skill of a programmer who can teach themselves.

You are given this file. **Do not modify it** — just read what it promises:

```
# token_source.py   -- GIVEN TO YOU, DO NOT MODIFY
class TokenSource:
    def __init__(self, data):
        self._data = data
        self._i = 0
    def has_next(self):        # True if another integer is still available
        return self._i < len(self._data)
    def next_token(self):      # returns the next integer, then moves forward
        value = self._data[self._i]
        self._i = self._i + 1
        return value
```

Its contract, in plain words:

```
src.has_next()    ->  True if an integer is still waiting, else False
src.next_token()  ->  hands you the next integer and steps forward one
```

You are also given `main.py`. **Fill in only the while loop** so that `total` ends up holding the sum of every integer the source hands out. (Rule: call `next_token()` only when `has_next()` is True.)

```
# main.py
from token_source import TokenSource

def main():
    src = TokenSource([4, 8, 15, 16, 23])
    total = 0

    # ---- WRITE ONLY THIS WHILE LOOP ----


    # ------------------------------------
    print(total)

main()
```

```
Expected:
  For [4, 8, 15, 16, 23]   ->   66
  For an empty source []   ->   0
```

Your while loop:

```
_____
_____
```

S3b (follow-up). Keeping everything else the same, change your loop so it prints the **largest** integer the source hands out instead of the sum. What should it print if the source is empty? Note your choice and your reasoning.

```
_____
```